Evaluación de Unidad

1. Datos Generales

Asignatura	Álgebra Lineal
Ciclo	1 A
Unidad	1
Resultado de aprendizaje de la unidad	Aplica y resuelve analíticamente utilizando diversos métodos sistemas de ecuaciones lineales, matrices y determinantes, aplicaciones lineales según diversos criterios y utilizando diversas técnicas para la resolución de problemas con ayuda de software matemático, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Nombre del Docente	Edwin René Guamán Quinche
Nombre del Estudiante	Jinmy Javier Vega Yaguana
Fecha	08 / 11 / 2026
Horario	3h
Github	https://github.com/jinmyvega0-stack/ALGEBRA_LINEAL_1_CICLO.git

2. Objetivo:

Demostrar la capacidad de identificar, modelar matemáticamente y resolver un problema práctico utilizando el álgebra lineal aplicando sistemas de ecuaciones lineales o matrices.

3. Contexto

En este proyecto analizamos la reconstrucción progresiva de imágenes mediante las operaciones básicas de matrices del Álgebra Lineal. Elegimos este tema porque nos pareció una forma muy gráfica de aplicar la materia: representamos una imagen como un arreglo

de números y visualizamos su estructura desde una matriz reducida de 1×1 (el color promedio global) escalando progresivamente sus dimensiones ($2 \times 2{,}4 \times 4{,}8 \times 8$ ...) hasta mostrar la resolución completa de 1024×1024. Para llevar a cabo esta aproximación matricial utilizamos multiplicación de matrices por bloques de unos, sumas y restas para el ajuste de los canales de color, y determinantes mediante cofactores para la fundamentación teórica.

Este tema es importante en la Ingeniería en Computación porque constituye el principio básico con el que los programas redimensionan gráficos e incrementan la resolución de pantalla. Con la realización de este proyecto aprendemos a manipular matrices en Python mediante la librería NumPy, a descomponer una imagen en sus tres canales independientes (Rojo, Verde y Azul) y a comprobar cómo las operaciones matriciales vistas en clase tienen una aplicación directa en el procesamiento digital de gráficos.

4. **Desarrollo de las operaciones**

4.1. Planteamiento del problema

Para este trabajo vamos a simular cómo cambia la resolución de una imagen y cómo se modifican sus colores usando operaciones de álgebra lineal. El procedimiento parte de un píxel inicial de 1×1 para ampliarlo a 2×2 y luego a 4×4 mediante multiplicaciones con matrices auxiliares de duplicación y por bloques. Al final, ajustaremos la intensidad de los colores haciendo sumas y restas de escalares directamente en ciertas posiciones de las matrices.

Matrices con las que vamos a trabajar:

- **Píxel inicial (1×1):**

$$M = (120)$$

- **Matriz auxiliar de duplicación (J_2) y su traspuesta (J_2^T):**

$$J_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad J_2^T = (1 \quad 1)$$

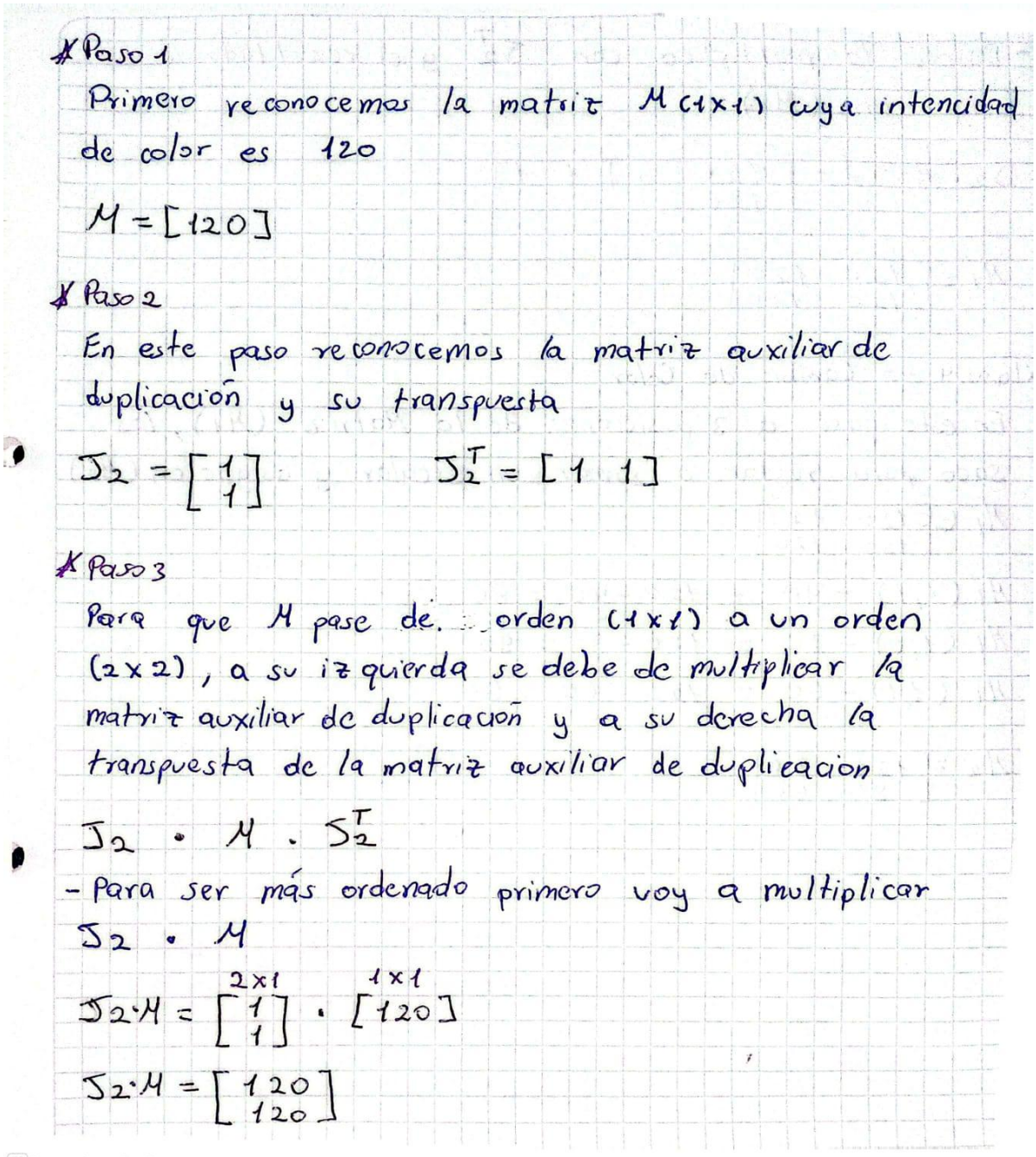
- **Matriz auxiliar por bloques (J_b) y su traspuesta (J_b^T):**

$$J_b = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, \quad J_b^T = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

Operaciones que se van a realizar:

1. **Multiplicación de matrices:** Para escalar las dimensiones aplicando $M_1 = J_2 \cdot M \cdot J_2^T$ y $M_3 = J_b \cdot M_2 \cdot J_b^T$.
2. **Transposición de matrices:** Obtención de las matrices traspuestas J_2^T y J_b^T.
3. **Suma y resta de escalares:** Cambios manuales en los valores numéricos de las posiciones elegidas para simular el cambio de color de los píxeles.

4.2. Procedimiento paso a paso



Escaneado con CamScanner

Figura 1. Paso 1, 2,3 reconocimiento de datos para calcular las matrices

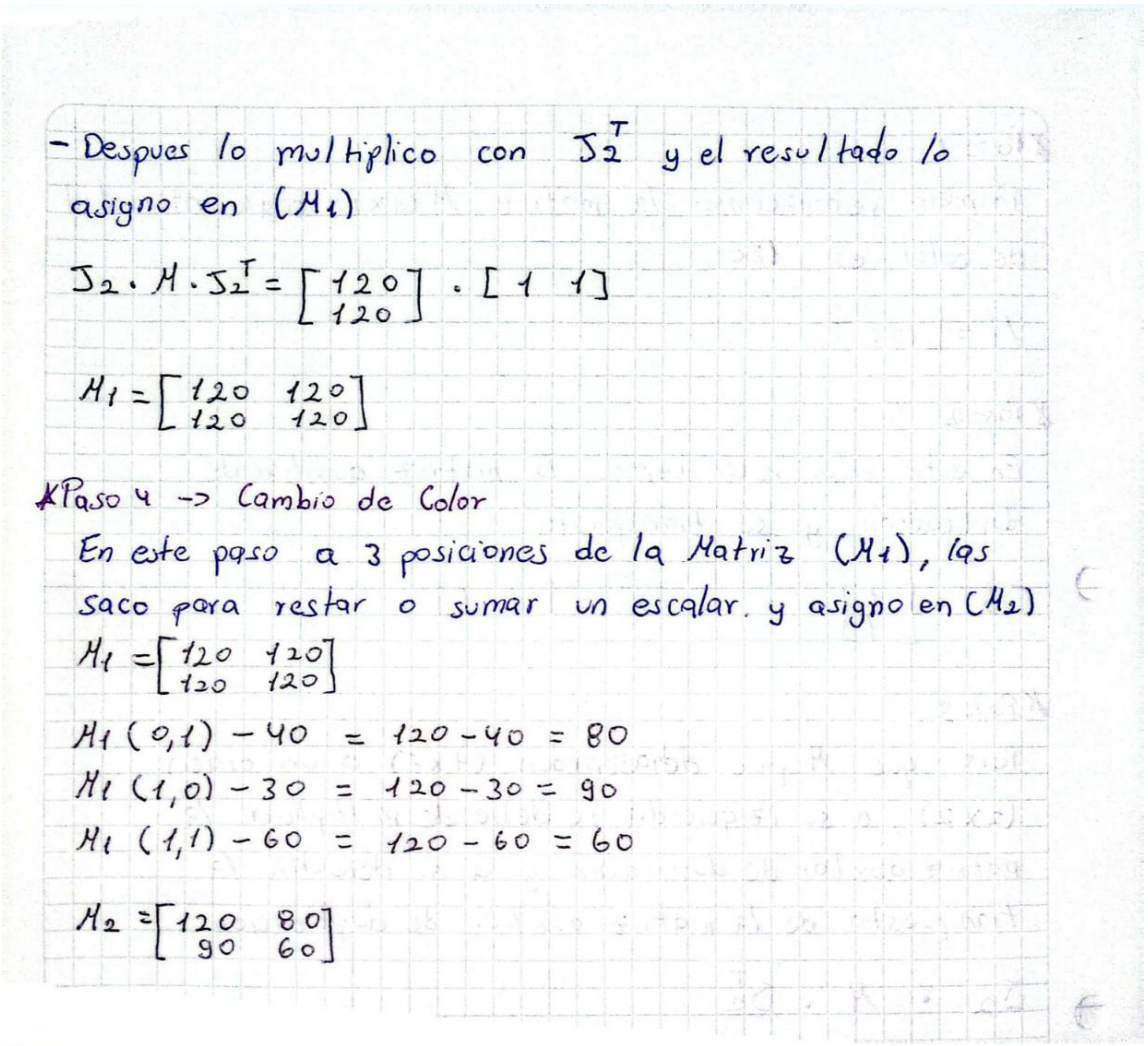
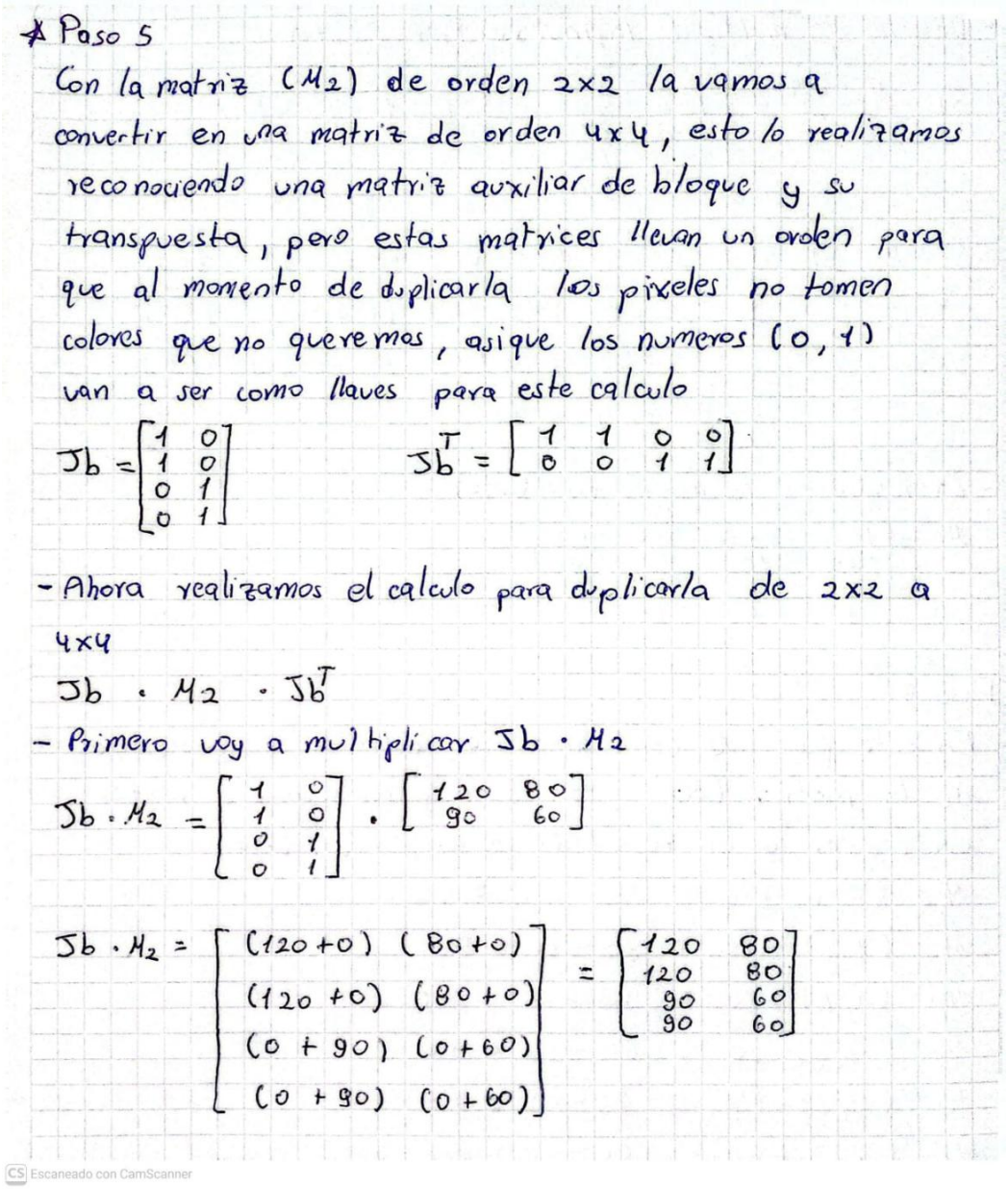


Figura 2. A continuación del paso 3. Paso 4 cambio de color a la matriz 2 * 2



* Paso 5

Con la matriz (M_2) de orden 2x2 la vamos a convertir en una matriz de orden 4x4, esto lo realizamos reconociendo una matriz auxiliar de bloque y su transpuesta, pero estas matrices llevan un orden para que al momento de duplicarla los pixeles no tomen colores que no queremos, asique los numeros (0, 1) van a ser como llaves para este calculo

$$Jb = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \qquad Jb^T = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

- Ahora realizamos el calculo para duplicarla de 2x2 a 4x4

$$Jb \cdot M_2 \cdot Jb^T$$

- Primero voy a multiplicar $Jb \cdot M_2$

$$Jb \cdot M_2 = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 120 & 80 \\ 90 & 60 \end{bmatrix}$$

$$Jb \cdot M_2 = \begin{bmatrix} (120+0) & (80+0) \\ (120+0) & (80+0) \\ (0+90) & (0+60) \\ (0+90) & (0+60) \end{bmatrix} = \begin{bmatrix} 120 & 80 \\ 120 & 80 \\ 90 & 60 \\ 90 & 60 \end{bmatrix}$$

Escaneado con CamScanner

Figura 3. Paso 5 duplicación de matriz 2 * 2 a 4 * 4

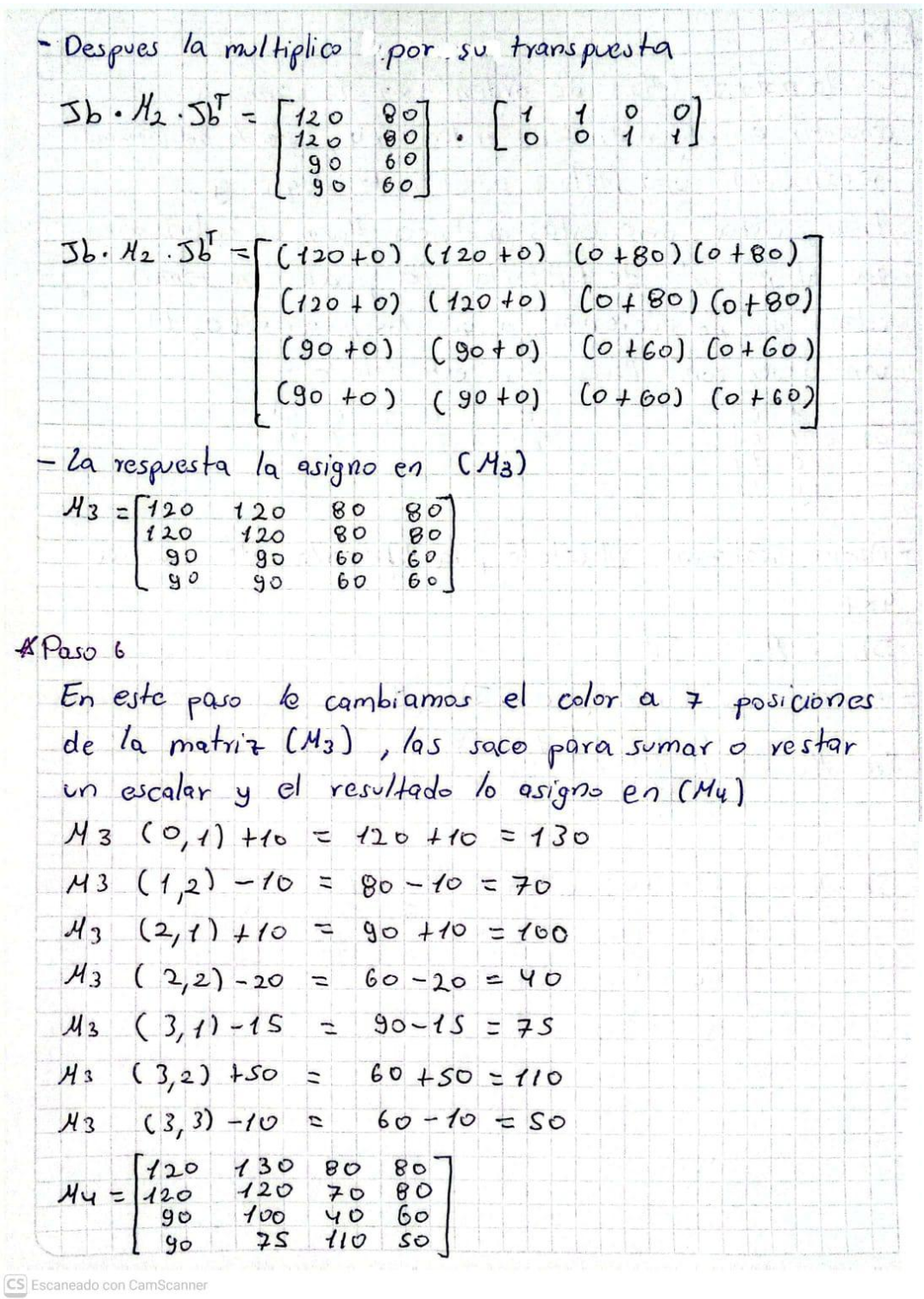


Figura 4. Resultado del paso 5. Paso 6 cambio de color a la matriz 4 * 4

4.3 Verificación de resultados



Figura 5. Verificación: determinante de M_2 (2×2) y de una submatriz de 3×3.

5. **Procedimiento / Metodología**

5.1 Descripción de la aplicación

¿En qué consiste el proyecto y qué problema resuelve?

Es una herramienta en Python, hecha en Google Colab, que convierte una imagen en matrices y las procesa con operaciones del Álgebra Lineal. La hicimos porque la teoría de matrices y determinantes suele quedarse en números sueltos sobre el papel. Con el programa se ve qué pasa con los datos de una imagen al duplicar sus píxeles para generarse y cómo eso afecta propiedades como el determinante.

¿A quién va dirigida?

Está dirigida a estudiantes y docentes de carreras técnicas que quieran ver una aplicación concreta del Álgebra Lineal en el procesamiento de imágenes.

¿Cuál es el objetivo principal?

Mostrar con un ejemplo práctico que duplicar los píxeles de una imagen repite filas, que esas filas repetidas son linealmente dependientes y que por eso el determinante de la matriz ampliada es 0 (matriz singular). También mostrar que, si se modifican valores concretos de la matriz, como hicimos a mano con M_4, desaparecen las filas repetidas.

¿Cómo se conecta con el Álgebra Lineal?

El programa usa estos temas de la materia:

- **Multiplicación de matrices:** la usamos en $J \cdot M \cdot J^T$ para duplicar el tamaño de la imagen píxel por píxel.

- **Transpuesta de una matriz:** al multiplicar por J^T a la derecha se duplican las columnas, igual que J a la izquierda duplica las filas.

- **Cofactores y determinantes:** el programa calcula determinantes por desarrollo de Laplace (cofactores) para comprobar si dan 0 cuando hay filas repetidas.

- **Suma y resta de matrices:** sumamos y restamos valores en los canales de color para cambiar el aspecto de la imagen.

5.2 Descripción de la aplicación

Categoría	Herramienta / Tecnología	Versión / Detalle
Lenguaje de programación	Python	3.10.12
Entorno de desarrollo (IDE)	Google Colab	Entorno interactivo en la nube
Librerías / Frameworks	NumPy, Matplotlib, PIL	NumPy: 1.26.4 Matplotlib: 3.7.1 PIL: 9.4.0
Plataforma de ejecución	Web / Nube	Ejecución en navegador a través de servidores de Google Colab

5.3 Funcionalidades implementadas

Funcionalidad 1: Carga de la imagen y conversión a matriz RGB

Descripción: sube una imagen a Google Colab, la redimensiona a 1024×1024 y la convierte en una matriz con tres canales de color.

Aplica: representación de datos en matrices (arreglo tridimensional).

Funcionalidad 2: Determinante por cofactores

Descripción: reduce una matriz de 4×4 a simetrices de 3×3 y 2×2 con código propio y muestra cada término en la consola.

Aplica: desarrollo por cofactores y determinantes.

Funcionalidad 3: Escalado de la imagen y duplicación con $J \cdot M \cdot J^T$

Descripción: genera la imagen en tamaños desde 1×1 hasta 1024×1024 promediando bloques de píxeles. La función duplicar_resolucion_canal agranda una matriz con $J \cdot M \cdot J^T$, repitiendo filas y columnas.

Aplica: multiplicación de matrices, transpuesta y dependencia lineal entre filas (determinante 0).

Funcionalidad 4: Ajuste de color

Descripción: suma 20 al canal rojo, resta 30 al azul y multiplica el verde por 1.05.

Aplica: suma y resta de matrices y multiplicación por un escalar.

Funcionalidad 5: Visualización

Descripción: muestra las 11 imágenes en una cuadrícula con matplotlib.

Aplica: representación gráfica de matrices como imágenes en color.

5.4 Capturas de pantalla

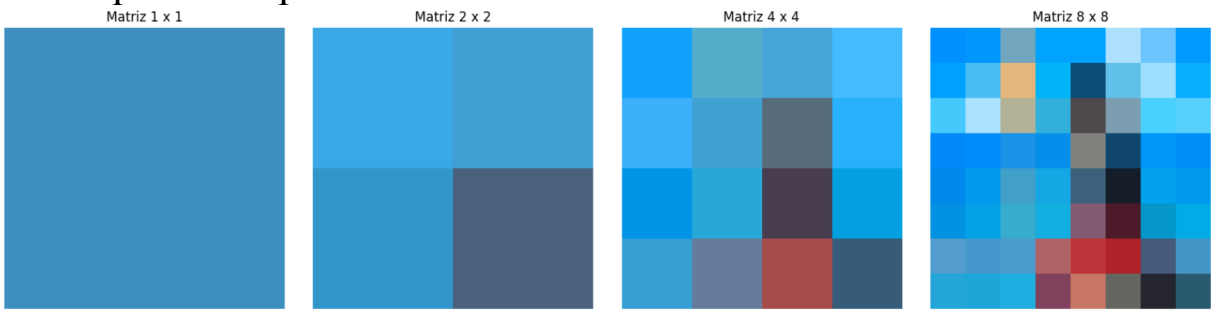


Figura 6. Matrices de 1×1, 2×2, 4×4 y 8×8: la imagen empieza a definirse.



Figura 7. Matrices de 16×16, 32×32, 64×64 y 128×128: se acercan a la imagen original.



Figura 8. Matrices de 256×256, 512×512 y 1024×1024; la de 1024×1024 ya tiene el ajuste de color aplicado.

5.5 Código fuente (fragmentos clave)

```
//Compresión de imagen por bloque

def promediar_bloques(img_full, target_dim):
    factor = 1024 // target_dim
    res = np.zeros((target_dim, target_dim, 3))
    for i in range(target_dim):
        for j in range(target_dim):
            bloque = img_full[i*factor:(i+1)*factor, j*factor:(j+1)*factor]
            res[i, j, 0] = np.mean(bloque[:, :, 0])
            res[i, j, 1] = np.mean(bloque[:, :, 1])
            res[i, j, 2] = np.mean(bloque[:, :, 2])
```

```
    return res


//Duplicación de los pixeles
def duplicar_resolucion_canal(canal_previo):
    n = canal_previo.shape[0]
    T = np.zeros((2 * n, n))
    for i in range(n):
        T[2 * i, i] = 1.0
        T[2 * i + 1, i] = 1.0
    return T @ canal_previo @ T.T
// Asignación de color a los pixeles
canal_R = secuencia_imagenes[-1][:, :, 0]
canal_G = secuencia_imagenes[-1][:, :, 1]
canal_B = secuencia_imagenes[-1][:, :, 2]


matriz_ajuste_R = np.ones((1024, 1024)) * 20.0
matriz_ajuste_B = np.ones((1024, 1024)) * 30.0


canal_R_ajustado = np.clip(canal_R + matriz_ajuste_R, 0, 255)
canal_G_ajustado = np.clip(canal_G * 1.05, 0, 255)
canal_B_ajustado = np.clip(canal_B - matriz_ajuste_B, 0, 255)


img_final_ajustada = np.zeros((1024, 1024, 3))
img_final_ajustada[:, :, 0] = canal_R_ajustado
```

```
img_final_ajustada[:, :, 1] = canal_G_ajustado

img_final_ajustada[:, :, 2] = canal_B_ajustado



secuencia_imagenes[-1] = img_final_ajustada
```

Link/Google Colab
https://colab.research.google.com/drive/1NHlN-4P4-XIl7iIGTE1tNMfvXHuWr0mj?usp=sharing

6. Resultados

- **¿Los resultados del cálculo manual coinciden con los de la aplicación? ¿Qué diferencias existen y a qué se deben?**

Sí, en lo que pudimos comparar. A mano calculamos el determinante de M_2 (2×2) y de una submatriz de 3×3, y en los dos casos dio exactamente 0. El programa da el mismo resultado porque usa la misma lógica de cofactores; la diferencia es que hace todo de una vez y muestra cada paso en pantalla.

- **¿Cuáles fueron las mayores dificultades que encontraste al resolver el problema manualmente? ¿Y al programarlo?**

A mano: lo que más nos costó fue entender la matriz auxiliar de duplicación y armar bien la matriz por bloques. Al principio no era obvio cómo acomodar los ceros y unos para que la multiplicación repitiera cada píxel sin descuadrar las dimensiones.

Al programar: usamos IA para armar la base del código, así que la dificultad no fue escribir el código sino entender qué hacía por dentro. Tuvimos que revisar cómo se aplica $J_2 \cdot M \cdot J_2^T$ y ajustar el programa para que mostrara los cofactores paso a paso, como en la teoría. Lo más largo fue entender cada línea para poder explicarla.

- **¿En qué situaciones reales de la computación o ingeniería se aplica el tema que estudiaste?**

Escalado de imágenes: repetir filas y columnas con $J \cdot M \cdot J^T$ equivale al escalado por vecino más cercano. Los escaladores más finos, como los de los televisores 4K o los videojuegos, interpolan entre píxeles vecinos, pero también se basan en operaciones con matrices.

Filtros de fotos: sumar, restar o multiplicar por un escalar los canales R, G y B es la base de los ajustes de brillo y color de editores como Photoshop y de los filtros de redes sociales.

- **¿Qué aprendiste al construir la aplicación que no hubieras aprendido solo con teoría?**

Construir la aplicación nos mostró cómo funcionan las imágenes por dentro: una foto es un arreglo de números, y agrandarla o cambiarle el color es operar con esos números. En la pizarra las matrices parecen no tener sentido práctico; al ejecutar el código vimos que la multiplicación de matrices cambia el tamaño de la imagen y que sumar o restar valores en RGB cambia los colores. Eso nos hizo ver para qué sirve el Álgebra Lineal.

7. Conclusiones

1. Las matrices auxiliares de duplicación (J_2 y J_b) y sus transpuestas permiten ampliar una matriz de 1×1 a 2×2 y de 2×2 a 4×4 con una sola multiplicación, $J \cdot M \cdot J^T$. La posición de los unos y los ceros define cómo se repiten las filas y las columnas.
2. Calculamos a mano el determinante de una matriz de 2×2 y de una submatriz de 3×3 de las matrices ampliadas, y en ambos casos dio 0. Esto confirma que son singulares: al duplicar se repiten filas, o quedan proporcionales, y hay dependencia lineal.

8. Referencias

[1] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed. New York, NY, USA: Pearson, 2018.

[2] The SciPy community, "NumPy Reference: Array objects and Linear Algebra (numpy.linalg)," NumPy Documentation, 2024. [Online].